

# Multi-Piano: LAN Multiplayer Terminal Piano

## CMPE487

Ömer Faruk Erzurumluoğlu    Berkay Keskin

Department of Computer Engineering

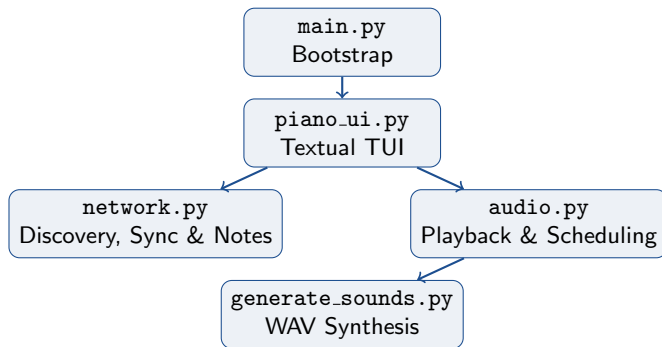
June 2026

- 1 Project Overview
- 2 System Architecture
- 3 Networking
- 4 Audio Pipeline
- 5 User Interface
- 6 Deployment & Future Work

**Multi-Piano** is a LAN-based collaborative piano application with a terminal user interface (TUI).

- Multiple players on the same network discover, host, and join lobbies.
- Notes are transmitted in near real time and played back with **clock-synchronised** audio.

**Goal:** demonstrate a lightweight multiplayer prototype combining discovery, mixed protocols, NTP-style clock sync, scheduled payout, and a full TUI.



We separate concerns into three network channels:

| Port  | Protocol | Format               | Role                        |
|-------|----------|----------------------|-----------------------------|
| 12488 | UDP      | JSON                 | Lobby discovery (broadcast) |
| 12487 | TCP      | length-prefixed JSON | Lobby management            |
| 12489 | UDP      | binary + JSON        | Note events & clock sync    |

- **Discovery** finds open sessions without prior configuration.
- **TCP** maintains player membership and state synchronisation.
- **UDP 12489** carries 15-byte note packets and NTP-style SYNC\_REQ/SYNC\_ACK messages.

## Client behaviour:

- Binds UDP port 12488.
- Sends DISCOVER broadcasts; listens for LOBBY\_REPLY responses.

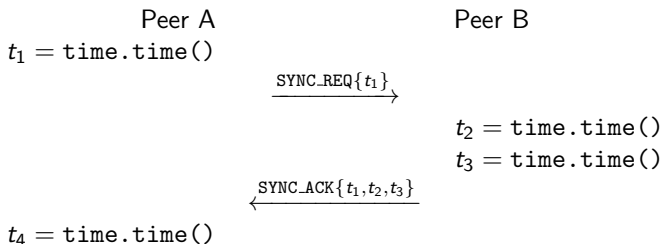
## Host behaviour:

- Replies to discoverers with lobby name, host IP, TCP port, and active player count.

## Join flow:

- 1 Client connects to (host\_ip, 12487).
- 2 Client sends {"type": "JOIN", "SENDER\_NAME": ..., "SENDER\_IP": ...}.
- 3 Host accepts, stores the socket, and appends the client IP to peer\_ips.
- 4 Host broadcasts STATE\_UPDATE with the current player list to all clients.
- 5 Both sides start a clock-sync loop (every 5 s) over UDP 12489.

**NTP-style exchange** (JSON on UDP 12489, every 5 s):



**Offset calculation** (peer clock – local clock):

$$\text{offset} = \frac{(t_2 - t_1) + (t_3 - t_4)}{2}$$

- Trip delays cancel; result is the pure clock difference.
- Both sides sync with each other; host syncs with every guest.
- Receiver converts a sender timestamp.



**Note packet** (15 bytes, little-endian, UDP 12489):

seq (u32) | play\_at (f64) | note\_id (u8) | action (u8) | velocity (u8)

**Sender** (on key press):

- Computes `play_at = time.time() + 30ms` and fires a local Timer for own audio.
- Broadcasts the packet with `play_at` embedded so every peer targets the same instant.

**Receiver — NoteScheduler:**

- Converts: `local_play_at = play_at - offset`.
- Inserts into a **monotonic priority queue**.
- Background thread sleeps until `local_play_at`, then plays the note.
- Events more than 200 ms overdue are discarded.

# Thank you for your attention

**Ömer Faruk Erzurumluoğlu**  
**Muhammet Berkay Keskin**

omer.erzurumluoglu@std.bogazici.edu.tr  
muhammet.keskin@std.bogazici.edu.tr